

TP6

September 18, 2025

1 Temas Tratados en el Trabajo Práctico 6

- Modelado de problemas en espacios de estado.
- Algoritmos de planificación hacia adelante y hacia atrás.
- Representación y solución de problemas descritos en lenguaje STRIPS.
- Algoritmo GRAPHPLAN.
- Planificación con restricciones de tiempo y recursos.
- Caminos críticos y tiempos de relajación.

1.1 Ejercicios Teóricos

1. ¿En qué tipo de algoritmos se basa un planificador para encontrar el mejor camino a un estado solución?

Un planificador se basa en algoritmos de búsqueda, aplicados sobre un modelo lógico del problema. Utiliza representaciones de estados, acciones y objetivos en forma de literales, y busca una secuencia de acciones que transforme el estado inicial en uno que cumpla el objetivo. Las acciones tienen precondiciones y efectos, y todo el proceso se formaliza con herramientas como STRIPS. Este enfoque permite al agente construir planes complejos de forma eficiente, sobre todo en entornos deterministas y totalmente observables.

2. ¿Qué tres elementos se encuentran dentro de una acción formulada en lenguaje STRIPS? Describa brevemente qué función cumple cada uno.

Una acción en lenguaje STRIPS tiene tres elementos principales:

- Nombre y parámetros: Identifica la acción y las variables que intervienen. Ejemplo: Volar(p, desde, hasta) indica que se trata de una acción de vuelo con un avión p desde un lugar a otro.
 - Precondiciones: Son las condiciones que deben cumplirse en el estado actual para que la acción pueda ejecutarse. Ejemplo: En(p, desde) Avión(p) indica que el avión debe estar en el lugar de origen.
 - Efectos: Describen cómo cambia el estado del mundo después de ejecutar la acción. Ejemplo: $\neg$ En(p, desde) En(p, hasta) indica que el avión ya no está en el lugar de origen y ahora está en el destino.
3. Describa las ventajas y desventajas de desarrollar un algoritmo de planificación hacia adelante y hacia atrás en el espacio de estados.

1. Planificación hacia adelante (forward search): Parte del estado inicial y aplica acciones disponibles hasta llegar a un estado que cumpla el objetivo.
 - Ventajas:
 - Es directa: el estado inicial está completamente definido.
 - Es fácil verificar qué acciones son aplicables.
 - Útil cuando se conocen bien las condiciones iniciales y se puede simular el entorno.
 - Desventajas:
 - Puede generar muchos estados irrelevantes para el objetivo.
 - Difícil mantener el enfoque en lo que realmente se quiere lograr.
2. Planificación hacia atrás (backward search): Parte del objetivo y busca acciones que puedan lograrlo, retrocediendo hasta llegar al estado inicial.
 - Ventajas:
 - Se enfoca en las acciones necesarias para cumplir el objetivo, evitando caminos inútiles.
 - Puede ser más eficiente cuando los objetivos son muy específicos.
 - Desventajas:
 - Es más complejo verificar si las precondiciones de las acciones se cumplen en algún estado posible.
 - Difícil de aplicar si no se conoce bien el estado inicial o si el dominio tiene muchas acciones posibles.
4. Considere el problema de ponerse uno mismo zapatos y medias. Aplique GRAPHPLAN a este problema y muestre la solución obtenida. Muestre el plan de orden parcial que es solución e indique cuántas linealizaciones diferentes existen para el plan de orden parcial.

Problema: ponerse uno mismo zapatos y medias.

- Tenemos 4 acciones:
 - – PonerseMedia(Izq)
 - – PonerseMedia(Der)
 - – PonerseZapato(Izq)
 - – PonerseZapato(Der)
- Con las siguientes precondiciones y efectos:
 - – PonerseMedia(Pie)
 - – * Precondición: Ninguna
 - – * Efecto: Media(Pie)
 - – PonerseZapato(Pie)
 - – * Precondición: Media(Pie)

- — * Efecto: Zapato(Pie)

Aplicación de GRAPHPLAN

GRAPHPLAN construye niveles alternando niveles de acciones y niveles de estados. Las acciones sólo pueden ejecutarse si sus precondiciones están en el nivel anterior y no hay conflictos entre acciones.

1. Nivel S0 (Estado inicial):

- $\neg$ Media(Izq)
- $\neg$ Media(Der)
- $\neg$ Zapato(Izq)
- $\neg$ Zapato(Der)

2. Nivel A0 (Acciones posibles):

- PonerseMedia(Izq)
- PonerseMedia(Der)

3. Nivel S1 (Efectos):

- Media(Izq)
- Media(Der)

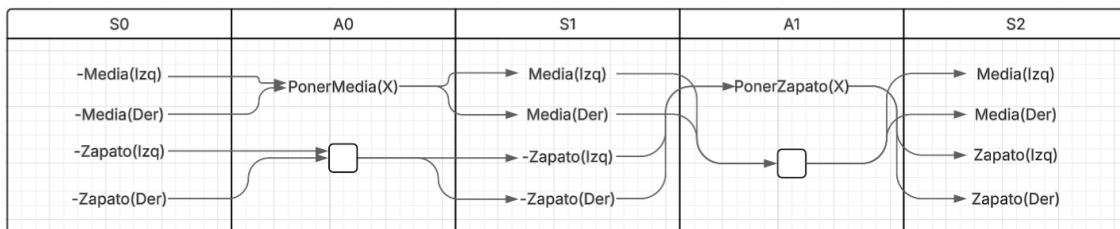
4. Nivel A1 (Acciones posibles):

- PonerseZapato(Izq) (porque Media(Izq) está disponible)
- PonerseZapato(Der) (porque Media(Der) está disponible)

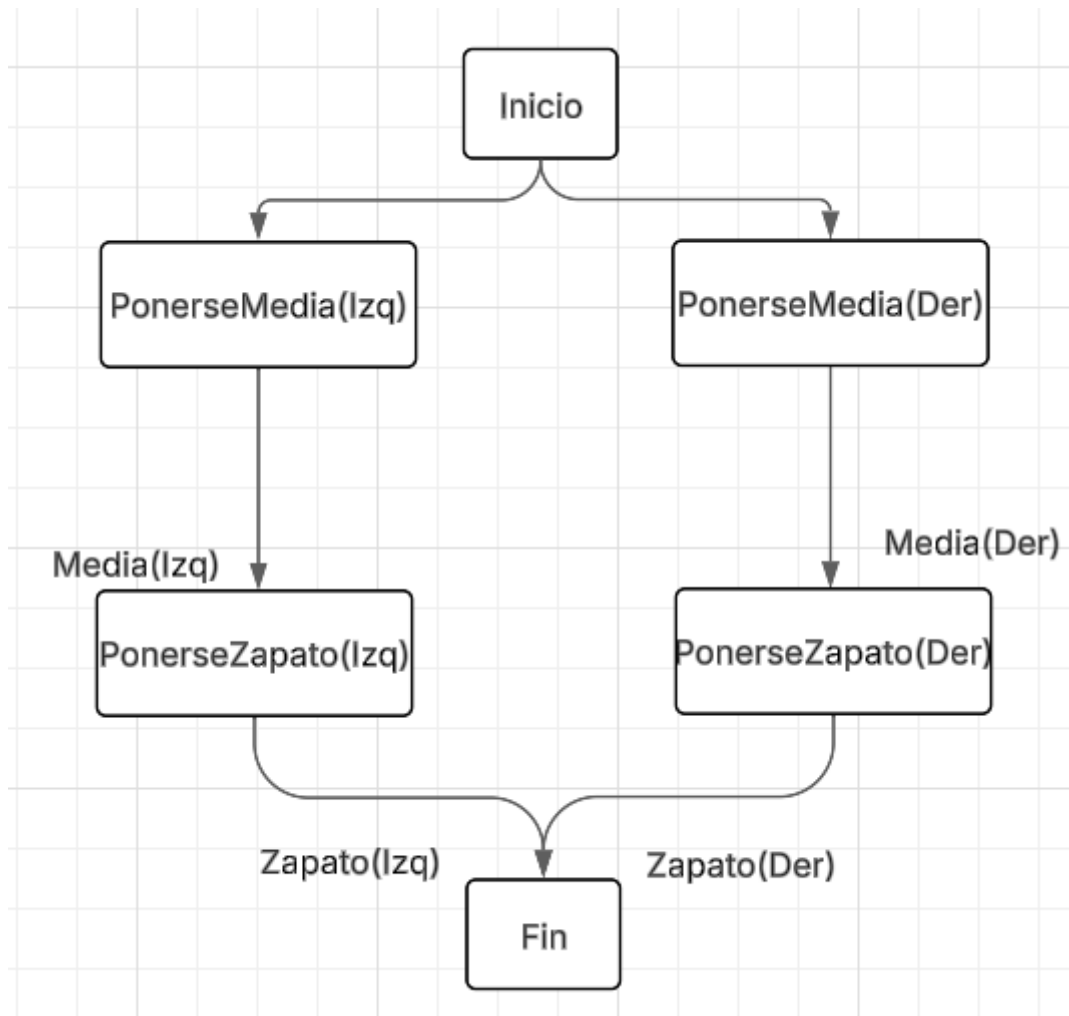
5. Nivel S2 (Efectos):

- Zapato(Izq)
- Zapato(Der)

Objetivo alcanzado.



El plan de orden parcial viene dado por la siguiente imagen:



Las acciones `PonerseMedia(Izq)` y `PonerseMedia(Der)` pueden ocurrir en cualquier orden entre sí, igual que `PonerseZapato(Izq)` y `PonerseZapato(Der)`, siempre y cuando se respeten las dependencias por pie.

Las linealizaciones posibles son 6 diferentes:

1. `Media(Izq) → Zapato(Izq) → Media(Der) → Zapato(Der)`
2. `Media(Izq) → Media(Der) → Zapato(Izq) → Zapato(Der)`
3. `Media(Izq) → Media(Der) → Zapato(Der) → Zapato(Izq)`
4. `Media(Der) → Zapato(Der) → Media(Izq) → Zapato(Izq)`
5. `Media(Der) → Media(Izq) → Zapato(Der) → Zapato(Izq)`
6. `Media(Der) → Media(Izq) → Zapato(Izq) → Zapato(Der)`

Conclusion: GRAPHPLAN encuentra un plan válido y completo. El plan de orden parcial asegura las dependencias mínimas, y las 6 linealizaciones muestran la flexibilidad del plan. Esto ilustra cómo la planificación en IA no solo encuentra soluciones, sino que también permite diferentes secuencias equivalentes de ejecución.

5. Se requiere ensamblar una máquina cuyas piezas están identificadas con las letras A, B, C, D y E. El tiempo que se tarda en ensamblar cada pieza es:

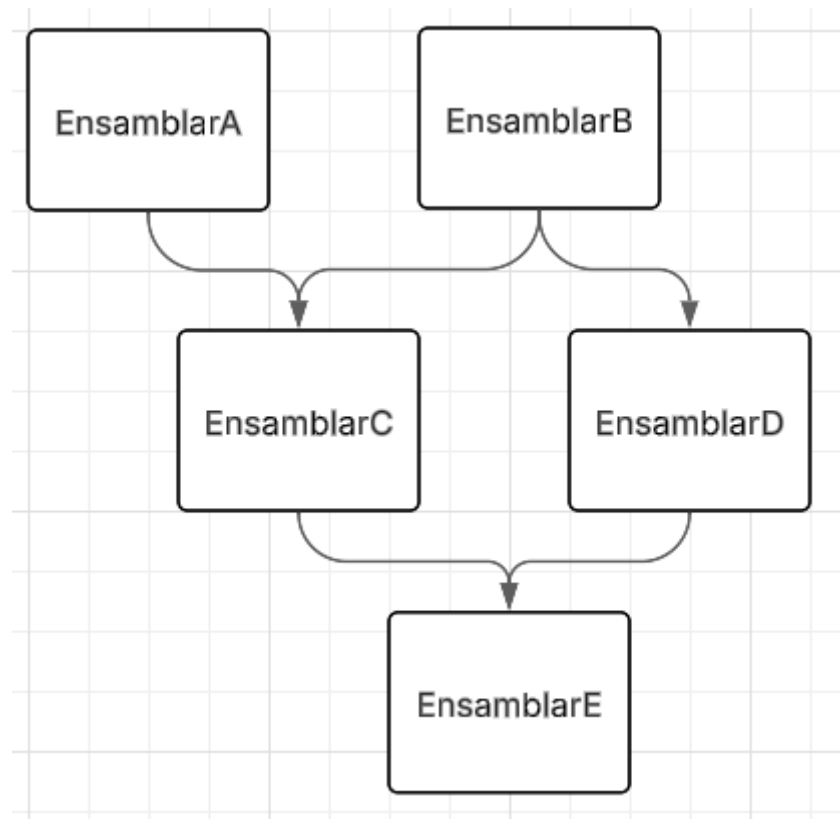
- A: 2 semanas
- B: 1 semana
- C: 4 semanas
- D: 3 semanas
- E: 5 semanas

El orden de ensamblaje de cada pieza requiere que:

- A esté realizado antes que C
- B esté realizado antes que C
- B esté realizado antes que D
- C esté realizado antes que E
- D esté realizado antes que E

Con esta información:

5.1 Arme el Plan de Orden Parcial.



5.2 Encuentre el Camino Crítico.

El camino critico se encuentra al hacer el analisis de la duracion total del proceso. Eso se hace sumando los tiempos maximos de los procesos que estan encadenados. El proceso C necesita de A y de B. El de mayor duraci3n es A. Luego el ensamble de E necesita de C y D, el de mayor duracion es C. Por ultimo se toma el tiempo de duracion de E para terminar el proceso.

$$2(A) + 4(C) + 5(E) = 11$$

Luego el camino critico es aquel que no tiene holguras, es decir que su retraso implica un retraso en el total de proceso.

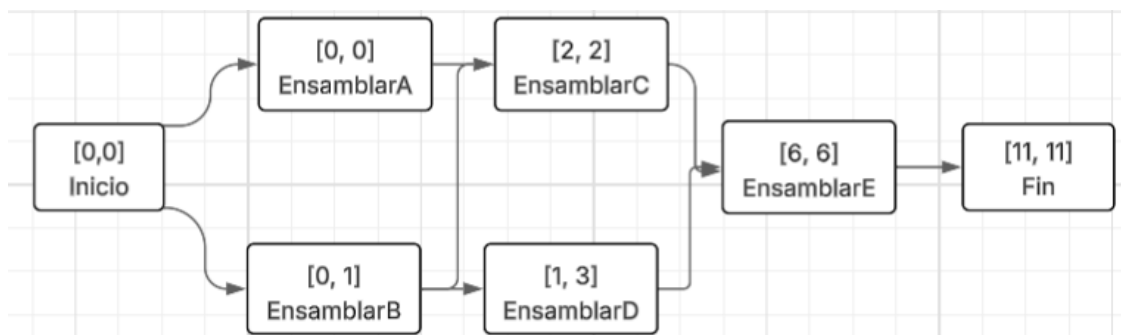
Camino cr3tico: **A → C → E**

Duraci3n total: **11 semanas**

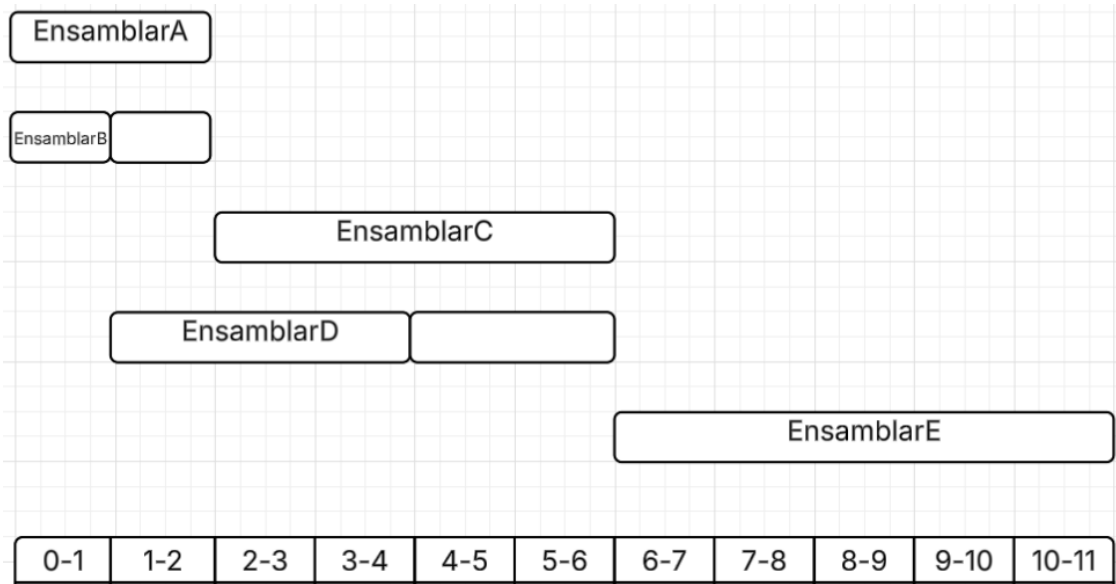
5.3 Encuentre los tiempos de relajaci3n.

Los tiempos de relajacion son aquellos que permiten que un proceso se retrase o comience luego sin retrasar a la operacion total. En el caso de este problema es el proceso B y el proceso D los cuales tienen holguras, de una y dos semanas respectivamente.

Tarea	Dur (sem)	ES	EF	LS	LF	Holgura
A	2	0	2	0	2	0
B	1	0	1	1	2	1
C	4	2	6	2	6	0
D	3	1	4	3	6	2
E	5	6	11	6	11	0



5.4 Dibuje un diagrama temporal indicando las tareas y los tiempos de relajaci3n encontrados.



1.2 Ejercicios de Implementación

6. Suponga que tiene un robot de oficina capaz de moverse y tomar y depositar objetos. El robot solo puede tener un objeto a la vez, pero puede conseguir una *caja* en la que depositar varios objetos. Suponga que programa al robot para *ir a la tienda* a comprarle un *café* y en el camino de vuelta tome una *carta* del *buzón* de la oficina para para que se la traiga junto con el café. Describa en lenguaje STRIPS:

6.1 El dominio del robot (nombre, predicados y acciones que puede hacer el robot).

6.2 El problema que se quiere resolver (estado inicial, estado objetivo y objetos del mundo representados).

6.3 Introduzca el código desarrollado en los puntos anteriores en el [planificador online](#) y obtenga el plan de acción que tomará el robot para cumplir lo solicitado.

[]: 6.1-

```
(define (domain robot-strips)
  (:predicates
    (habitacion ?hab)
    (objeto ?obj)
    (robot-en ?hab)
    (en ?obj ?hab)
    (libre)
    (llevando ?obj)
    (en-caja ?obj)
    (conectado ?a - hab ?b - hab)
  )

  (:action moverse
    :parameters (?from ?to)
```

```

:precondition (and (conectado ?from ?to) (robot-en ?from))
:effect (and (robot-en ?to)
              (not (robot-en ?from)))
)

(:action agarrar
:parameters (?obj ?hab)
:precondition (and (objeto ?obj) (habitacion ?hab)
                  (en ?obj ?hab) (robot-en ?hab) (libre))
:effect (and (llevando ?obj)
             (not (en ?obj ?hab))
             (not (libre)))
)

(:action soltar
:parameters (?obj ?hab)
:precondition (and (objeto ?obj) (habitacion ?hab)
                  (llevando ?obj) (robot-en ?hab))
:effect (and (en ?obj ?hab)
             (libre)
             (not (llevando ?obj)))
)

(:action meter_en_caja
:parameters (?obj ?hab)
:precondition (and (objeto ?obj) (habitacion ?hab)
                  (llevando ?obj) (robot-en ?hab)
                  (en caja ?hab))
:effect (and (en-caja ?obj)
             (libre)
             (not (llevando ?obj)))
)

(:action sacar_de_caja
:parameters (?obj ?hab)
:precondition (and (objeto ?obj) (habitacion ?hab)
                  (en caja ?hab) (en-caja ?obj) (robot-en ?hab)
                  (libre))
:effect (and (llevando ?obj)
             (not (en-caja ?obj))
             (not (libre)))
)
)
)

6.2-

(define (problem strips-robot-x-1)

```



```

(:domain robot-strips)
(:objects
  oficina tienda buzon
  cafe carta caja;
)
(:init
  (habitacion oficina)
  (habitacion tienda)
  (habitacion buzon)
  (objeto cafe)
  (objeto carta)
  (objeto caja)
  (robot-en oficina)
  (libre)
  (en cafe tienda)
  (en carta buzon)
  (en caja oficina)
  (conectado oficina tienda)
  (conectado tienda buzon)
  (conectado buzon oficina)
)
(:goal
  (and (en caja oficina)
        (en-caja cafe)
        (en-caja carta))
)
)

```

6.3 -

```

(moverse oficina tienda)
(moverse tienda buzon)
(agarrar carta buzon)
(moverse buzon oficina)
(meter_en_caja carta oficina)
(moverse oficina tienda)
(agarrar cafe tienda)
(moverse tienda buzon)
(moverse buzon oficina)
(meter_en_caja cafe oficina)
; cost = 10 (unit cost)

```

Los resultado no son los esperados, faltó tiempo para resolver mejor el lisp. Sabemos que la solucion es: Agarrar caja, moverse a tienda, soltar caja, agarrar cafe, ponerlo en caja, moverse a buzon, dejar caja, agarrar carta, ponerla en caja, agarrar caja, mover a oficina, dejar caja. Para eso debemos cambiar los costos de las acciones

2 Bibliografía

Russell, S. & Norvig, P. (2004) *Inteligencia Artificial: Un Enfoque Moderno*. Pearson Educación S.A. (2a Ed.) Madrid, España

Poole, D. & Mackworth, A. (2023) *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press (3a Ed.) Vancouver, Canada